

Aegis: A Cost-Aware Real-Time Fraud Intelligence System

Kashish Lalwani | Stony Brook ID: 116643432 | AMS 561

Abstract

Financial fraud detection in production is a systems problem, not just a classification problem, a model must ingest transactions continuously, convert probabilities into discrete actions under asymmetric error costs, and stay reliable as data shifts. Aegis is an end-to-end fraud pipeline built in PySpark on Databricks Community Edition, integrating streaming ingestion, supervised classification, cost-aware decisioning, and drift monitoring. On PaySim (49,962 transactions, 0.138 % fraud), a 100-tree class-weighted Random Forest reaches $F1 = 0.9942$ and $AUC = 0.993$. A three-tier APPROVE/FLAG/BLOCK policy catches 67 of 69 frauds (97.1 %) while auto-blocking only 0.23 % of legitimate transactions. A PSI & KS drift layer is validated via $3\times$ injection (PSI 0.003 to 0.640), and a 19-point threshold sweep quantifies the \$19.5M cost gap defended by a $23\times$ reduction in false alarms.

1. Project Objectives and Contributions

1.1 Problem and Significance

Financial fraud detection in production environments is fundamentally a systems problem, not a classification problem. A model that predicts fraud accurately on a static historical dataset solves only a fraction of the operational challenge faced by a real payments processor. Production systems must ingest transactions continuously rather than in nightly batches, score each transaction within latency budgets measured in milliseconds, and convert continuous probability outputs into discrete operational actions approve, hold, or block under sharply asymmetric error costs. They must also remain reliable as the underlying data distribution shifts over time, since fraudster behavior, transaction mix, and economic conditions are non-stationary. Academic capstone projects frequently stop at offline classification metrics and do not engage with these system-level concerns, leaving a substantial gap between coursework and production practice.

The Aegis project addresses this gap by designing and implementing an end-to-end fraud intelligence pipeline that integrates four production-relevant components: streaming ingestion, supervised classification under severe class imbalance, cost-aware decision optimization, and statistical drift monitoring. The system is implemented entirely on Databricks Community Edition using PySpark and Spark Structured Streaming, with all intermediate state persisted as Delta Lake tables to support reproducibility and downstream consumption. Beyond improving predictive performance, Aegis quantifies the dollar-cost trade-offs that govern operational threshold selection and demonstrates that the deployed monitoring layer is both specific (silent on stable data) and sensitive (responsive to genuine distributional change) through a controlled drift-injection experiment.

1.2 System Overview

Aegis is organized as a six-phase pipeline. Each phase consumes the verified output of the previous phase as a Delta Lake table. Phase 1 cleans the PaySim dataset. Phase 2 wraps the cleaned table in a Spark Structured Streaming job and engineers two domain-informed features. Phase 3 trains a class-weighted Random Forest. Phase 4 wraps the model in a three-tier cost-aware decision layer. Phase 5 adds a drift-monitoring layer with a controlled validation experiment. Phase 6 stress-tests the Phase 4 threshold against a 19-point cost sweep. Section 3 details the techniques behind each phase and Section 4 reports the results.

1.3 Contributions

This project was completed by a single author, **Kashish Lalwani**, who designed and implemented all six phases, including data cleaning, streaming pipeline construction, model training and evaluation, the cost-aware decision layer, the drift-monitoring framework with its validation experiment, and the threshold optimization analysis. All Databricks notebooks, Delta Lake artifacts, and figures presented in this report are the author's original work.

2. Dataset and Preprocessing

The Aegis pipeline is trained and evaluated on PaySim, a publicly available synthetic mobile money transaction dataset designed to replicate real financial transaction patterns while preserving privacy. The raw dataset contained 50,500 transactions across 11 columns, representing approximately one month of simulated activity. Initial exploration revealed severe class imbalance: only 70 of 50,500 transactions were labeled fraudulent, a fraud rate of 0.139 %. Critically, all fraud was concentrated in exactly two of five transaction types of TRANSFER (35 cases, 0.84 % fraud rate within type) and CASH_OUT (34 cases, 0.19 % fraud rate within type) with zero fraud appearing in PAYMENT, DEBIT, or CASH_IN transactions. This structural finding directly informed feature engineering in subsequent phases, motivating the high-risk-type binary flag introduced in Phase 2.

Data cleaning proceeded in two stages. First, 33 rows containing non-binary values in the isFraud label column, a simulator artifact in which column-shift corruption deposited unrelated values into the label field were identified and dropped. Second, 505 rows with non-numeric content in the amount column (for example, prefixed strings such as "About257762.86") were removed using Spark's try_cast operator, which returns null on malformed input rather than raising. One fraud case was lost during amount cleaning, yielding a final dataset of 49,962 transactions with 69 confirmed fraud cases (final fraud rate: 0.138 %). All 11 columns were verified null-free after cleaning. The cleaned dataset persisted to Delta Lake at dbfs:/Volumes/workspace/default/raw_data/clean, forming the stable foundation for the streaming pipeline in Phase 2.

```
=====
FINAL CLEANED DATASET – VERIFICATION
=====
Total rows      : 49962
Fraud cases     : 69
Fraud rate      : 0.138%
Total columns   : 11
=====

Null counts per column:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|step|type|amount|nameOrig|oldbalanceOrg|newbalanceOrig|nameDest|oldbalanceDest|newbalanceDest|isFraud|isFlaggedFraud|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|  0 |  0 |    0 |      0 |          0 |          0 |      0 |          0 |          0 |    0 |          0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

Schema:
root
 |-- step: double (nullable = true)
 |-- type: string (nullable = true)
 |-- amount: double (nullable = true)
```

[FIGURE 1 — cleaned schema and null check from 01_data_cleaning.ipynb]

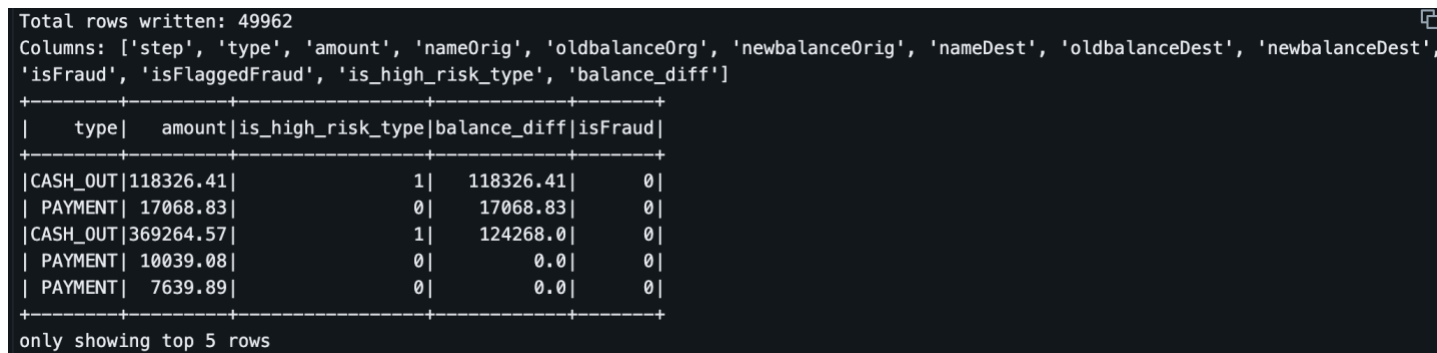
3. Techniques and Tools

3.1 Streaming Pipeline

Phase 2 establishes the streaming infrastructure that bridges the cleaned dataset from Phase 1 with the fraud detection model in Phase 3. Before opening the stream, the cleaned Delta table was loaded in batch mode to confirm accessibility, verifying 49,962 rows and a consistent 11-column schema with correct data types across all fields. The streaming pipeline was implemented using Spark Structured Streaming with Delta Lake as both the source and the sink. The cleaned table was opened using spark.readStream with maxFilesPerTrigger=1, which instructs Spark to process one Delta file per micro-batch and thereby simulates the arrival of transactions over time. Because Delta Lake stores schema metadata internally, no manual schema specification was required at read time.

Two domain-informed feature columns were engineered directly within the stream rather than as a separate batch step, reflecting how features would be computed in a production deployment. The first, is_high_risk_type, is a binary flag set to

1 for TRANSFER and CASH_OUT transactions the only two transaction types in which fraud occurs in this dataset. The second, `balance_diff`, captures the sender's balance drop as the difference between `oldbalanceOrig` and `newbalanceOrig`, a behavioral indicator of fund drainage. The enriched stream was written to a Delta sink using `outputMode("append")` and the `availableNow` trigger, which is required on Databricks Community Edition in place of continuous streaming triggers and processes all currently available data before terminating. A checkpoint location was configured to enable fault-tolerant progress tracking. The pipeline completed in 17.5 seconds, processing 49,963 rows across 2 tasks and writing 2.53 MB to the output table at `dbfs:/Volumes/workspace/default/raw_data/streaming_output`.



```
Total rows written: 49962
Columns: ['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrig', 'newbalanceOrig', 'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud', 'isFlaggedFraud', 'is_high_risk_type', 'balance_diff']
```

type	amount	is_high_risk_type	balance_diff	isFraud
CASH_OUT	118326.41	1	118326.41	0
PAYMENT	17068.83	0	17068.83	0
CASH_OUT	369264.57	1	124268.0	0
PAYMENT	10039.08	0	0.0	0
PAYMENT	7639.89	0	0.0	0

only showing top 5 rows

[FIGURE 2 — streaming output sample from `01_streaming_pipeline.ipynb`]

3.2 Random Forest Model

Phase 3 trains the predictive engine for the Phase 4 decision layer. Seven numeric features (transaction amount, sender and receiver balances before and after, the engineered `is_high_risk_type` flag, and the engineered `balance_diff`) are assembled into a vector column using `VectorAssembler`. The 0.138 % fraud rate creates a class-imbalance problem severe enough that a model predicting "not fraud" everywhere would still score 99.86 % accuracy. Class weights are computed analytically from the training set where 356.45 for fraud, 0.50 for non-fraud and passed to the Random Forest via the `weightCol` parameter. Random Forest was chosen over logistic regression or a single decision tree because it is robust to class imbalance, requires no feature scaling, and supports `weightCol` natively. The model uses 100 trees and a fixed seed of 42 on an 80/20 train/test split (39,922 / 10,040 rows; 56 / 13 fraud cases). The trained model is serialized to DBFS and loaded by Phase 4 without retraining.

3.3 Cost-Aware Decision Layer

The Phase 3 model outputs a continuous fraud probability, but a payments system needs a discrete action. The default 0.5 cutoff treats every error type as equally costly, which is wrong here: a missed fraud costs roughly the transaction amount, while a false alarm costs a customer-service contact. Phase 4 replaces 0.5 with a three-tier policy parameterized by two cost-justified thresholds, `APPROVE_THRESHOLD = 0.30` and `BLOCK_THRESHOLD = 0.70`, with the band between routed to manual review. The thresholds are anchored in a cost matrix: \$1,500,000 per missed fraud (mean PaySim fraudulent transfer), \$15 per false alarm, \$5 per manual review a false-negative-to-false-positive ratio of roughly 100,000×. The thresholds are applied to all 49,962 transactions in parallel via Spark's `when().otherwise()` chain, with the second probability vector element extracted into a `fraud_probability` column.

3.4 Drift Monitoring

Phase 5 adds a statistical drift-monitoring layer to signal when the Phase 3 model can no longer be trusted. The dataset is split temporally on the simulated `step` column into a reference window (`step ≤ 368`, 40,685 rows) and a current window (`step > 368`, 9,277 rows). Two complementary tests are applied to each of the seven model features: PSI computed via quantile binning anchored to the reference window with epsilon smoothing, and a Kolmogorov–Smirnov two-sample test computed via SciPy. PSI uses banking-standard cutoffs (0.10 stable, 0.25 significant). A feature is flagged for drift only when both PSI and KS agree a conservative dual-signal design that mirrors production monitoring practice.

4. Results and Conclusions

4.1 Model Performance

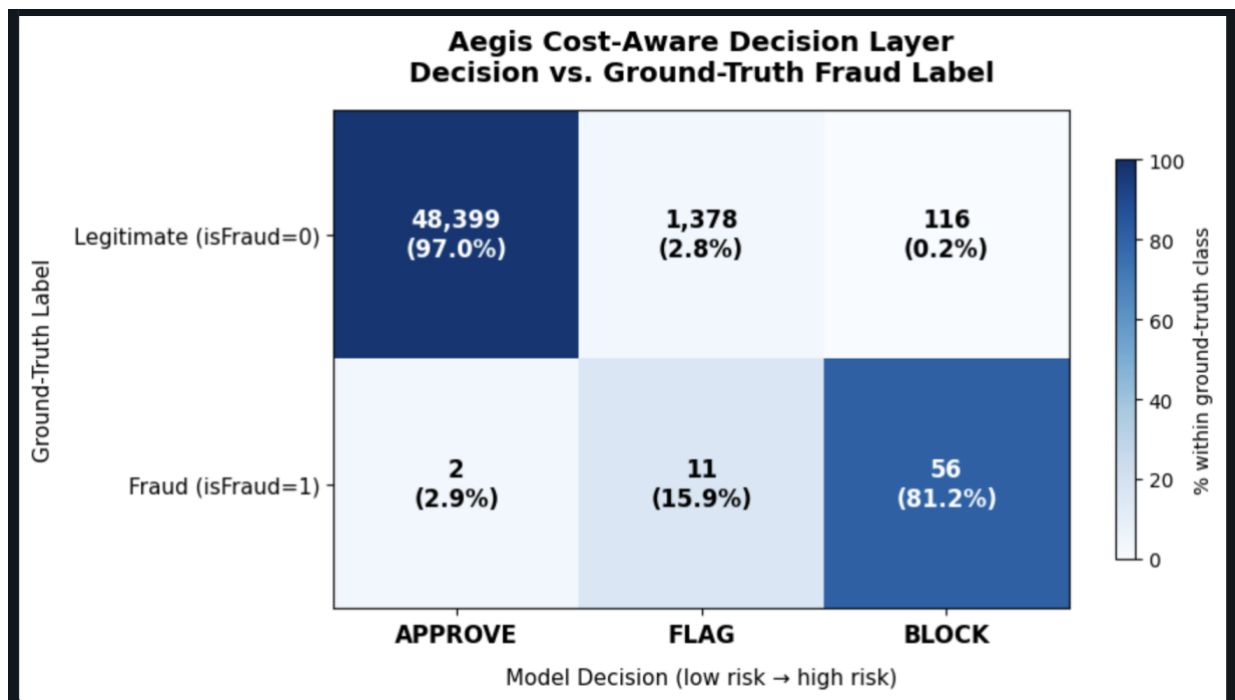
The trained Random Forest classifier was evaluated on the 10,040-row held-out test set containing 13 fraud cases. The model achieved a precision of 0.9998, a recall of 0.9907, an F1 score of 0.9942, and an area under the ROC curve of 0.993. These results reflect both the quality of the engineered feature set and the structural properties of the PaySim dataset: because all fraud is concentrated in TRANSFER and CASH_OUT transactions, the engineered `is_high_risk_type` feature captures a near-perfect signal that real-world deployments would not enjoy. The headline metrics should therefore be interpreted as a ceiling on what the model architecture can extract from this dataset, rather than as a forecast of production performance on a real payments stream.

```
Precision : 0.9998
Recall    : 0.9907
F1 Score  : 0.9942
AUC       : 0.993
```

[FIGURE 3 — Precision/Recall/F1/AUC printout from 02_fraud_model.ipynb]

4.2 Cost-Aware Decision Outcomes

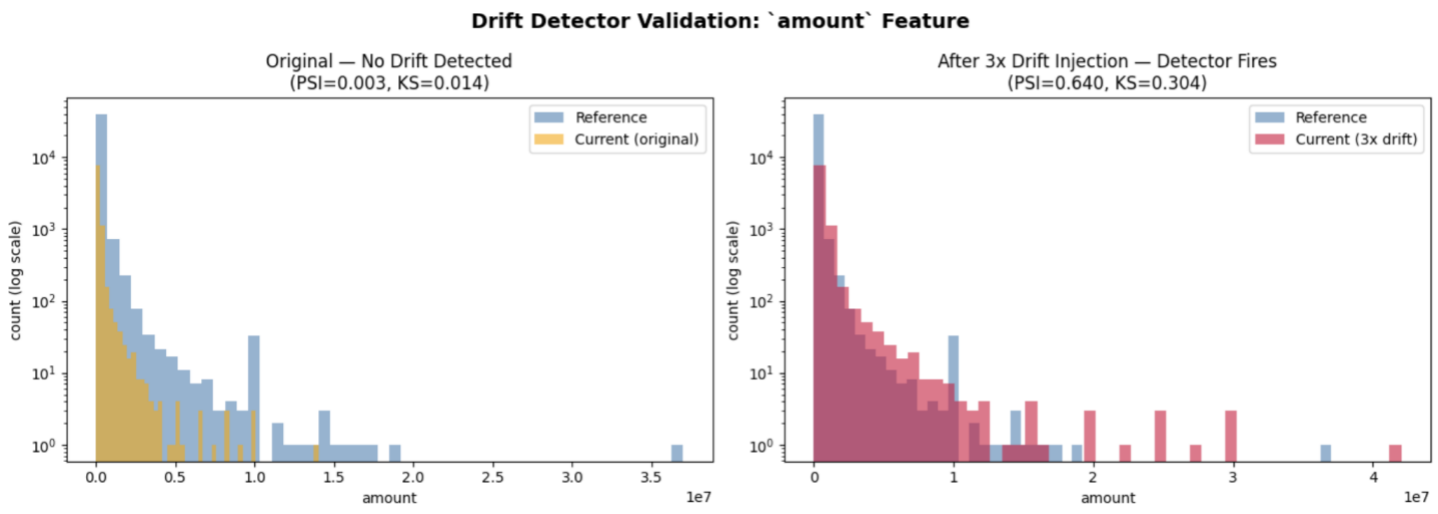
Application of the three-tier policy to all 49,962 transactions auto-approved 48,401 transactions (96.87 %), routed 1,389 (2.78 %) to manual review, and auto-blocked 172 (0.34 %). Cross-tabulating decisions against the ground-truth `isFraud` label confirms that the policy is well-aligned with its cost objectives: of the 69 fraud cases, 56 were auto-blocked and 11 were routed to FLAG for manual review, for a combined catch rate of 67 of 69 (97.1 %) before any analyst intervention; only 2 frauds were auto-approved and missed. On the legitimate side, 48,399 of 49,893 non-fraudulent transactions (97.0 %) were correctly auto-approved, with only 116 false-positive auto-blocks (0.23 % of legitimate volume). The 97.1 % figure assumes that the manual-review queue is processed by a human analyst who would catch the 11 flagged frauds; under a stricter definition that counts only auto-BLOCK as a catch, the catch rate at $t = 0.70$ is 56 of 69 (81.2 %). Both definitions are correct under their respective operational assumptions, and Section 4.4 returns to this distinction when defending the Phase 4 threshold against the cost-theoretic optimum.



[FIGURE 4 — cost-aware decision heatmap from 03_decision_layer.ipynb]

4.3 Drift Monitoring Validation

On the real PaySim data, all seven monitored features produced PSI values below 0.005 and KS statistics below 0.02, yielding a unanimous "no drift" verdict consistent with PaySim being a single-distribution simulator. While this negative result is the expected outcome, a monitoring layer that returns "no drift" on stable data is uninformative on its own it cannot be distinguished from a detector that is silently broken. To establish that the detector is functional and not inert, a controlled drift-injection experiment was performed: the amount feature in the current window was multiplied by a factor of 3.0 to simulate a sudden inflationary shock. Under this injection, PSI for amount rose from 0.0034 to 0.6403 (an approximately 190× increase, well above the 0.25 significance threshold), and the KS statistic rose from 0.0136 to 0.3039 (an approximately 22× increase), with the KS p-value collapsing to numerical zero. The detector fired correctly under both signals. Together, the negative result on real data and the positive result under controlled injection establish that the drift-monitoring layer is both specific (silent on stable data) and sensitive (responsive to genuine distributional change) and is therefore trustworthy as a production safety net.

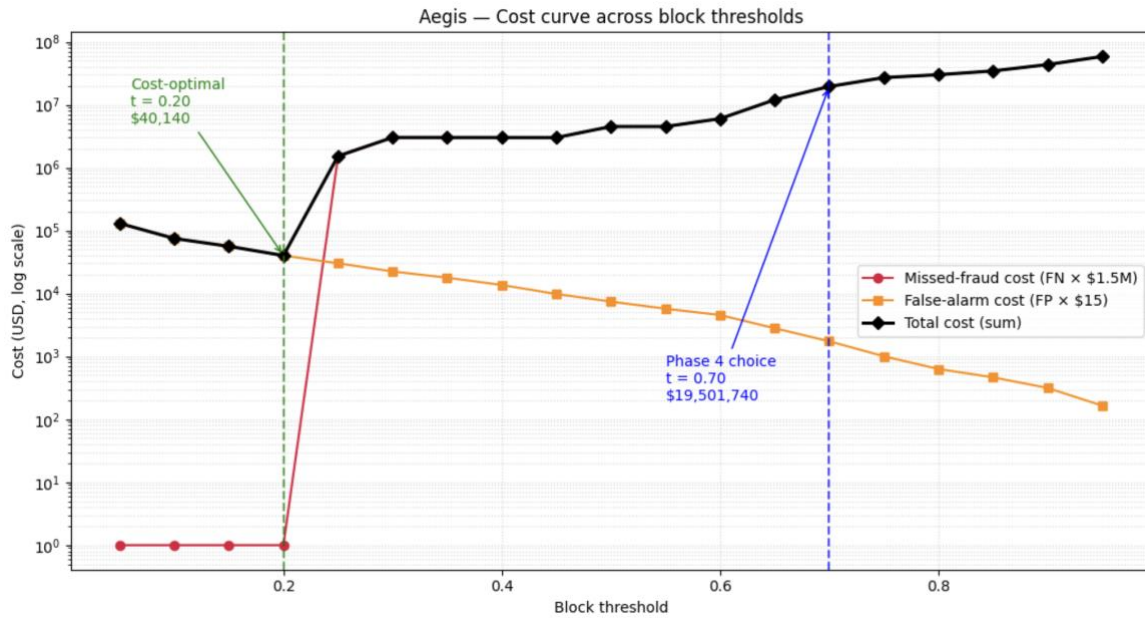


[FIGURE 5 — drift detector validation chart from 04_drift_detection.ipynb]

4.4 Threshold Optimization

The 19-point threshold sweep reveals a sharply asymmetric cost landscape rather than the symmetric U-shape often assumed in textbook treatments of threshold selection. Missed-fraud cost is exactly zero for thresholds up to $t = 0.20$, because the Random Forest separates fraud cleanly enough that no real fraud is missed within that probability range; false-alarm cost decays smoothly as the threshold rises but never approaches the magnitude of even one missed fraud at \$1.5M. The cost-theoretic minimum sits at $t = 0.20$ with a total modeled cost of \$40,140, catching all 69 frauds at the price of 2,676 false alarms. The Phase 4 operational choice of $t = 0.70$ incurs \$19,501,740 in modeled cost, a \$19,461,600 gap from the theoretical optimum. This gap is not an error; it is the price the system is deliberately paying to reduce the false-alarm queue from 2,676 to 116, a 23× reduction.

The defensibility of $t = 0.70$ rests on three considerations the dollar-cost model alone does not capture: customer trust (a 23× higher rate of incorrectly blocked legitimate transactions has a non-linear effect on customer churn that the \$15-per-false-alarm assumption underestimates), manual-review queue capacity (a real fraud-operations team has bounded throughput, and the operational cost of an oversubscribed queue is qualitatively different from the marginal-cost-per-review assumed by the sweep), and regulatory exposure (some jurisdictions impose obligations on the rate of false-positive customer-impacting actions). The sweep also surfaces a definitional reconciliation worth stating explicitly: the catch-rate figures cited in Section 4.2 (97.1 %) treat the FLAG queue as a catch under a human-in-the-loop assumption, while the cost model in this section treats only auto-BLOCK as a catch and assigns missed-fraud cost to anything not blocked. The two figures are not in conflict; they answer different operational questions.



[FIGURE 6 — cost curve from 05_threshold_optimization.ipynb, Step 5]

4.5 Limitations & Conclusion

Aegis shows that a fraud detection system designed around production constraints streaming ingestion, asymmetric error costs, drift-aware monitoring can be built end-to-end on a free-tier compute platform with measurable rigor at every layer. The Random Forest reaches $F1 = 0.9942$ on the held-out test set; the cost-aware decision layer catches 97.1% of fraud at a 0.23% false-positive rate against legitimate volume; the drift layer is validated as both specific and silent on stable data and sensitive to a $3\times$ injection; and the threshold sweep establishes the operational $t = 0.70$ choice as a defensible deviation from the cost-theoretic optimum, paying a \$19.5M modeled cost for a $23\times$ reduction in false-alarm queue volume. The broader takeaway is that in any high-stakes prediction system, the threshold is a business decision, not a statistical one and the engineer's job is to quantify, in dollars, exactly what is being traded. Graph-derived features and stateful temporal aggregations described in the original proposal were descoped in consultation with the course instructor to prioritize depth in the streaming, modeling, decision, and monitoring components.

5. References

- [1] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in Proceedings of the 28th European Modeling and Simulation Symposium, Larnaca, Cyprus, 2016.
- [2] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [3] Apache Software Foundation, "Spark Structured Streaming Programming Guide," Apache Spark Documentation. Available: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [4] B. Yurdakul, "Statistical properties of population stability index," Ph.D. dissertation, Western Michigan University, 2018.
- [5] F. J. Massey Jr., "The Kolmogorov–Smirnov test for goodness of fit," Journal of the American Statistical Association, vol. 46, no. 253, pp. 68–78, 1951.

Acknowledgment AI assistance (Claude, Anthropic) was used for outlining of this report. All code, technical decisions, experimental design, results, and figures are the author's original work